Estatly: Interview Guide

Everything a recruiter or interviewer is likely to ask about Estatly, in one place: what it is, how it is built, how sign-in and permissions work, how data stays fresh, the bugs you found and fixed, and what you would improve. Read it once end to end, then use the self-test.

Matches the code after the November 2026 review (auth, dashboard, clients, team, settings, viewings, properties, cleanup and profile branches).

Contents

- 1. The 60-second pitch
- 2. Architecture
- 3. Authentication flow
- 4. Permissions: three layers
- 5. Database security
- 6. The data layer
- 7. Validation and errors
- 8. Feature highlights
- 9. UI, accessibility and performance
- 10. Bugs you found and fixed
- 11. Trade-offs and what you would improve
- 12. If they say: show me the code
- 13. Interview questions
- 14. Vocabulary
- 15. Self-test

1. The 60-second pitch

Say this out loud until it sounds like you, not like a script.

"Estatly is a CRM for a real estate agency in Dubai. Agents add listings, track clients through a sales pipeline and schedule viewings. Managers approve listings, and admins manage the team and the agency settings.

It is built with React, TypeScript and Vite, with TanStack Query for server state, React Hook Form and Zod for forms, Tailwind and shadcn for the UI, and Supabase for the database, auth and file storage.

The part I am proudest of is security: there are three roles, and every rule is enforced in the database with row level security and triggers, not just hidden in the UI. Recruiters can try it with demo accounts for each role, and the database protects those accounts so one visitor can never lock out the next."

The numbers to know

What	Detail
Roles	Admin, manager, agent
Main pages	Dashboard, Properties, Viewings, Clients, Agents, Managers, Settings, Profile
Stack	React 19, TypeScript, Vite, TanStack Query 5, React Router 7, React Hook Form, Zod 4, Tailwind 4, shadcn, Supabase
Live demo	One demo account per role, password shared on the login page

Questions to expect right after the pitch

- Why Supabase and not your own backend? *Auth, Postgres, storage and RLS in one service, so I could focus on the product and still enforce security in the database.*
- What was the hardest part? *Permissions: making sure every rule is enforced by the database, then making the UI agree with it.*
- What would you do differently? See section 11.

2. Architecture

Folders by feature

Folder	Holds
pages/	One thin file per route: it only places the feature component
features/<name>/	Components and hooks for one feature: clients, viewings, properties...
api/	Every Supabase call. Components never call Supabase directly
lib/	Shared helpers: validation, errors, constants, permissions, the action() helper
components/	Shared UI: DataRenderer, forms fields, layout, shadcn ui
hooks/	Shared hooks: useDebounce, useStepBackFromEmptyPage

How a request flows

Layer	Job	Example
Page	Places the feature	pages/Clients.tsx
Component	Markup only	ClientsBoard
Page hook	State and handlers	useClientsBoard
Data hook	Cache and refresh	useClients, useMoveClient
API	Validate, query, shape	apiClients.ts
Database	Enforce the rules	RLS, triggers

Rules you followed

- **Dependency direction:** features use api and lib; lib never imports api or features. You fixed two places that broke this (counting helpers, buildPropertyPayload).
- **Logic in hooks, markup in components.** Every big page has a hook: useClientsBoard, useTeamMemberList, useViewingsTable, usePropertiesList.
- **Filtering on the server**; grouping only one screen needs goes in that screen's hook; components only render.
- **Shape data at the boundary:** the API turns database shapes into app shapes once (viewing_count instead of viewings[0].count).

3. Authentication flow

This is the flow interviewers ask about most. Know it in order.

Signing in

```
// api/apiAuth.ts
const { data, error } = await supabase.auth
  .signInWithPassword({ email, password })
if (error) throw toRequestError(error)

// Deactivated accounts are signed straight back out
const { data: profile } = await supabase
  .from("user_profiles").select("is_active")
  .eq("user_id", data.user.id).single()

if (!profile?.is_active) {
  await supabase.auth.signOut()
  throw new Error("Your account is not active...")
}
```

- Supabase checks the password and returns a session (access and refresh tokens, stored by the client).
- Then the profile is checked: a deactivated person is signed out at once. The database also blocks them everywhere through get_user_role() (section 5).

```
// features/auth/useLogin.ts
onSuccess: async () => {
  await queryClient.fetchQuery({
    queryKey: ["user"],
    queryFn: getCurrentUserApi,
    staleTime: 0,
  })
  navigate("/dashboard", { replace: true })
},
```

- **The user (with role) is loaded before entering the app**, so the sidebar never renders without a role.
- **staleTime: 0** forces a real fetch. A visit to /login before signing in cached null for the user; without it, that null would be reused and you would bounce back to login. A real bug you fixed.
- replace: true: Back does not return to the login page.

Who is signed in

```
// api/apiAuth.ts
export async function getCurrentUserApi() {
  const { data: session } = await supabase.auth.getSession()
  if (!session.session) return null

  const { data } = await supabase.auth.getUser()
  ...
  const { data: profile } = await supabase
    .from("user_profiles").select("*")
    .eq("user_id", data.user.id).single()

  return { ...data.user, user_profile: profile }
}
```

- **getSession** reads the stored session (fast, no request). It answers "is anyone here?".
- **getUser** asks the server to verify the token. It is the one to trust: a stored session can be edited or expired.
- The profile row is attached, so the role is always next to the user.

```
// features/auth/useUser.ts
useQuery({
  queryKey: ["user"],
  queryFn: getCurrentUserApi,
  staleTime: 5 * 60 * 1000,
})
```

- One cached user for the whole app. Every component that calls useUser() shares it: one request, many readers.
- 5 minutes stale time: the user rarely changes, and login, logout and profile updates refresh it themselves.

Guarding routes

```
// components/layout/ProtectedRoute.tsx
if (isLoading) return <Loader />
if (!isAuthenticated) return <Navigate to="/login" replace />
return <>{children}</>
```

- Wraps the whole app layout. While the session is checked, a loader; signed out, a redirect.
- **<Navigate> during render**, not useEffect plus navigate(): no flash of the protected page, and Back does not loop.

```
// App.tsx
<Route path="/settings" element={
  <RoleProtected allowed={["admin"]}>
    <Settings />
  </RoleProtected>
} />
```

- RoleProtected sends the wrong role to /dashboard. It is user experience: the database still refuses their requests (section 5).

Signing out

```
// features/auth/useLogout.ts
onSuccess: () => {
  queryClient.removeQueries()
  navigate("/login", { replace: true })
},
```

- **removeQueries()** empties the whole cache. On a shared computer, the next person must never see the last person's clients, even for a second.

Forgot password

- sendPasswordResetApi emails a one-time link to /reset-password (redirectTo uses window.location.origin, so it works locally and in production).
- Opening the link signs the person in for that purpose, and ResetPasswordForm calls the same changePasswordApi as the profile page.
- **No user enumeration:** the page says the same thing whether or not the email exists, so it cannot be used to discover accounts.

Demo accounts

- The demo password ships in the frontend on purpose, so recruiters can sign in as each role.
- Two database triggers (sql/03-demo-accounts.sql) lock those accounts: their role, active status, email and password cannot change. One visitor can never lock out the next.
- The profile page hides the password form for them, only to avoid a confusing error. The database is the real guard.

Interview line: *"The UI hides what you cannot do; the database makes it impossible."*

4. Permissions: three layers

Layer	Where	Purpose
1. Routes	App.tsx, RoleProtected	Who may open a page
2. UI	lib/permissions.ts, usePropertyPermissions	Which buttons each role sees
3. Database	RLS policies, triggers, get_user_role()	What is actually allowed. The only layer that cannot be bypassed

Layers 1 and 2 are for a clean experience. Anyone can open the browser console and call Supabase directly, so only layer 3 is security. That sentence is the answer to most permission questions.

Who can do what

Action	Admin	Manager	Agent
Add listing (pending)	Yes	Yes	Yes
Approve and publish	Yes	Yes	No
Edit listing	Any	Any	Own
Under offer, sold, rented	Yes	Yes	Own
Delete listing	Yes	No	No
See clients and viewings	All	All	Own
See agents page	Yes	Read-only	No
Managers page, settings	Yes	No	No
Create, edit, deactivate people	Yes	No	No

lib/permissions.ts

```
export const can = {
  // RLS: admin and manager can update any property,
  // agents only their own
  editAnyProperty: (role) => ["admin", "manager"].includes(role),
  // RLS: admin only
  deleteProperty: (role) => role === "admin",
  ...
}
```

- One object answers every "can this role..." question. Components never compare role strings themselves.
- Each comment says which rule the database also enforces, so the two layers can be checked against each other.

```
// features/properties/usePropertyPermissions.ts
const isOwner = property.agent_id === user?.id
return {
  canEdit: can.editAnyProperty(role)
    || (role === "agent" && isOwner),
  canPublish: can.publishProperty(role),
  ...
}
```

- Combines the role with ownership for one listing. The menu, the card and the details page all ask this one hook.

5. Database security

Row level security (RLS)

- Every table has policies: SQL conditions Postgres checks on every row for every request, like "agents see only rows where agent_id = auth.uid()".
- **get_user_role()** returns the caller's role, or null for a deactivated person, so a deactivated account loses access everywhere at once without changing a single policy.
- The publishable key in the frontend is public on purpose. It only identifies the project; RLS decides what each signed-in user may do.

The silent block (your most important pattern)

```
const { data, error } = await supabase
  .from("clients").update(params)
  .eq("id", id).select("id")

if (error) throw toRequestError(error, "Client")

// RLS blocks updates silently: no error, zero rows changed
if (data.length === 0)
  throw new ForbiddenError("You can only edit your own clients")
```

- When RLS blocks an update or delete, Postgres returns **no error**; it simply changes zero rows.
- Without .select("id") and the length check, the user would see "Saved" while nothing changed. Every update and delete in the app uses this check.

Triggers

Trigger	Stops
handle_new_user	Creates every new account as an inactive agent: nobody can sign up into a role
protect_profile_columns	Non-admins changing role, is_active or email, even with a hand-made request
protect_demo_profiles / logins	Changes to the demo accounts

Your security audit (sql/01-security-audit.sql)

- Managers could delete properties: now admin only.
- A leftover policy let managers edit agent profiles: removed.
- Any signed-in user could overwrite anyone's avatar: storage policies now check the file's owner.

- Deactivated users could still upload images: the policy uses get_user_role().
- SECURITY DEFINER functions were callable by signed-out visitors: execute revoked; trigger functions never callable from the API.

Performance (sql/02-performance.sql): auth.uid() and get_user_role() wrapped in (select ...) run once per query instead of once per row; duplicate policies merged; indexes added on foreign keys used in joins.

6. The data layer

TanStack Query holds all server data: it caches it, refetches it and shares it between components. Only UI state (a sheet is open, which tab) lives in useState.

Tool	Where you use it	Why
queryKey	["viewings", params]	Everything the query depends on goes in the key; each combination is cached separately
invalidateQueries	Every mutation	Marks data stale; refetches what is on screen
enabled	useDashboard, useProperty	Dependent query: wait for the user, or skip a bad URL
staleTime	useUser 5 min; login 0	How long cached data counts as fresh
keepPreviousData	Paged lists	The old page stays on screen while the next loads
select	usePropertyOptions	Turns cached rows into select options inside the hook
setQueryData	View counter	Writes the new count into the cache, no refetch
fetchQuery	useLogin	Load and wait before navigating
removeQueries	Logout	Empty the cache
retry: shouldRetry	App.tsx	Never retry 4xx errors that will fail again

The invalidation rule (the source of most bugs you fixed)

When you change a table, find every query that reads it, including joins. A person's name is joined into clients, properties, viewings and the dashboard, so refreshPeople() invalidates all of them after a profile or team edit. Viewings are counted on client and team cards, so viewing mutations refresh those too.

Invalidation is cheap: only queries on screen refetch now; the rest are marked stale and refetch when visited.

The URL is the state

- Filters, sort, search and page live in the URL (useSearchParams), not in useState.
- Links can be shared, refresh keeps the filters, Back works. The params object is the query key.

- useStepBackFromEmptyPage: if a delete leaves ?page= past the end, Supabase answers 416 and the hook steps back a page. One effect that reacts to the server, whatever caused it.

7. Validation and errors

Two schemas per form

```
// The rules, on strings (used by the form)
export const clientFormSchema = z.object({ ... })

// Same rules plus conversion (used by the API)
export const clientSchema = clientFormSchema.transform(
  (v) => ({
    ...,
    phone: textOrNull(v.phone),
    budget_min: numberOrNull(v.budget_min),
  })
)

export type ClientFormValues = z.infer<typeof clientFormSchema>
```

- Inputs work with strings; the database wants numbers and null. The form uses the plain schema, the API the transform.
- **z.infer gives the output type.** A schema with a transform has different input and output types; that is why there are two.
- The same rules run in the browser (instant feedback) and in the API (nothing invalid is sent).

The action() helper

```
const { params, user } = await action({
  params: values,
  schema: clientSchema,
  authorize: true,
})
```

- Every API write starts here: validate with Zod, use the converted output, and optionally load the signed-in user for owner columns like agent_id.
- Owner and status columns are set by the API, never sent by the form, so the browser cannot choose them.

Errors with meaning

- toRequestError turns Supabase and Postgres codes into app errors with readable messages: 23503 "linked to other records", 23505 "already exists", PGRST116 not found, PGRST103 page past the end.
- Error classes carry an HTTP-like status (400, 401, 403, 404, 409, 416). shouldRetry skips retries below 500.
- Toasts show err.message directly, because the API already wrote it for humans.
- An ErrorBoundary around the app catches render crashes and shows a recovery page instead of a blank screen.

8. Feature highlights

One or two things worth mentioning per page. Pick the ones that match the question.

Dashboard

Four queries run in parallel with Promise.all. head: true counts viewings without downloading rows. Role and user id are in the query key, so an agent never sees an admin's cached numbers. Agents get their own scope through filters on top of RLS.

Properties

Approval workflow: new listings are always inserted as pending-approval by the API, and status only changes through its own action. Photos are resized in the browser and uploaded in parallel with Promise.all, which keeps their order, so the first photo stays the cover. View counts use a database function (views_count + 1 in one step) to avoid a race condition.

Viewings

The search runs on the joined property with an inner join (!inner), so non-matching viewings are dropped instead of returning property: null. Rescheduling and deleting are restricted in the query itself (.in("status", ...)), so a completed viewing cannot be changed even by a direct call. Viewings that happened are records: they get a status, never a delete.

Clients

A pipeline board with four stages. Each client's viewing count comes from viewings(count) in the same query, so there is no N+1 problem. Delete is offered only for clients with no viewings, and ON DELETE RESTRICT enforces it in the database.

Team

Agents and managers share one component with a role prop. Workload counts come from three counted joins, with filters on the joined rows (active clients, upcoming viewings). Creating an account in the browser swaps the admin's session out and back (see section 11).

Settings

One agency row. The form is mounted only after the data loads, because useForm reads defaultValues once. A save bar appears only when something changed (isDirty).

Profile

Name, photo and password. The old avatar is deleted after the new one is saved. refreshPeople() updates every page that shows your name or photo.

9. UI, accessibility and performance

Performance

- **Code splitting:** every page is lazy-loaded with React.lazy and Suspense, so the first load only downloads the login page.
- **Skeletons** in the shape of the real content, so nothing jumps when data arrives (no layout shift).
- Images resized before upload; lazy-loaded thumbnails; database indexes on joined foreign keys.

Accessibility

- Semantic HTML: section with aria-label for board columns, dl/dt/dd for label and value pairs, article for cards.
- Icon-only buttons have aria-labels that include the item name ("Actions for Thomas Mueller").
- Toggle chips use aria-pressed; loading states use role="status" with screen-reader text.
- Forms use noValidate so only Zod's messages show, with errors after a field is left (mode: onTouched).

Responsive patterns

- Desktop table and phone card both rendered, CSS shows one: no JavaScript screen check, no flash.
- repeat(auto-fill, minmax(15rem, 1fr)): a property grid with no breakpoints.
- min-w-0 and minmax(0, ...) so long text truncates instead of widening the page.
- Sticky save bar above the phone tab bar and the iPhone home indicator (env(safe-area-inset-bottom)).

10. Bugs you found and fixed

Use these as STAR stories: situation, task, action, result. Two or three are enough in one interview.

Year built was never saved

Every listing showed "Year built: Not set". The date picker stores an ISO date, and the payload called Number() on it. That is NaN, and JSON turns NaN into null, so the request succeeded silently. I moved the conversion into the Zod transform, where every other conversion lives.

A rename that did not reach other pages

Renaming an agent updated the team page, but clients, properties and the dashboard kept the old name, because they join it. I listed every query that joins user_profiles and wrote one refreshPeople() helper used by every person edit.

The approval step could be skipped

The property form sent its status with every save, and RLS lets agents update their own listing, so an agent could publish without approval. I removed status from the form: the API inserts pending-approval and status changes only through its own action.

The default viewing length was ignored

An admin set 60 minutes in Settings, but new viewings showed 30. useForm reads defaultValues once, before the settings had loaded. I applied the default with resetField when the settings arrive, which also keeps the form from counting as edited.

A comma broke the search

Searching "Marina, Dubai" errored. The text went straight into Supabase's .or() filter, where commas are syntax. I clean the input first, the same idea as never building SQL from strings.

Login bounced back to the login page

A visit to /login cached null for the user. After signing in, that null was reused and the guard sent me back. Fixed by fetching the user with staleTime: 0 before navigating.

A security audit

I reviewed every policy and found managers could delete properties, anyone could overwrite another person's avatar, and some database functions were callable by signed-out visitors. All fixed in one SQL migration.

11. Trade-offs and what you would improve

Saying what you would change shows judgment. Pick two.

Today	Better	Why it matters
Admin creates accounts in the browser by swapping sessions	A Supabase Edge Function with auth.admin.createUser and the service role key	Never touches the admin's session; checks the caller on the server
Approval enforced by the API	A trigger that rejects an agent setting published	RLS lets agents update their own listing, status included
Untyped Supabase client (as unknown as casts)	Generated types with createClient<Database>	Every query typed, no casts
Temporary passwords shared by hand	Invitation emails (inviteUserByEmail)	People set their own password
Photos left behind after a failed save or an edit	Delete removed files, or a cleanup job	Storage cost
No automated tests	Vitest for schemas and helpers, Playwright for sign-in and permissions	Catch regressions like the ones above

Interview line: *"It works and it is secure where it matters, but if I had a week I would move account creation to an Edge Function and add tests around permissions first."*

12. If they say: show me the code

Open files in this order. Each one tells part of the story.

#	File	What to point at
1	App.tsx	Lazy pages, ProtectedRoute, RoleProtected per route, QueryClient with shouldRetry
2	api/apiAuth.ts	Sign-in, is_active check, getSession vs getUser
3	features/auth/useLogin.ts	fetchQuery with staleTime: 0 before navigating
4	lib/permissions.ts	One source for UI permissions, comments tied to RLS
5	lib/sql/01-security-audit.sql	What you found and fixed in the database
6	api/apiClients.ts	action(), the join with a count, the silent RLS check
7	lib/validation.ts	Form schema vs transform schema
8	features/viewings/useViewings.ts	URL as state, query key, keepPreviousData
9	lib/refreshPeople.ts	The invalidation rule in one helper
10	components/common/DataRenderer.tsx	Generic component with a render prop: loading, error, empty, data

13. Interview questions

Q1. Walk me through what happens when a user signs in.

Supabase checks the password and returns a session. I then check the profile: if the account is deactivated I sign it straight back out. If it is active, I fetch the full user with their role before navigating, so the app never renders without a role. Routes are guarded by ProtectedRoute and, per page, RoleProtected.

Q2. How do you handle permissions?

Three layers. Routes decide which pages a role can open, a permissions object decides which buttons they see, and the database decides what is actually allowed with row level security and triggers. Only the database layer is security, because anyone can call the API directly.

Q3. What is row level security?

Postgres policies that run on every row for every request. For example, agents can only select clients where assigned_agent_id is their own id. It means the rules hold no matter which client sends the request.

Q4. RLS blocks an update. What does your app see?

No error: the update just changes zero rows. I select the id after every update and delete and throw a ForbiddenError when the result is empty, so the user never sees a false success.

Q5. Is it safe to have the Supabase key in the frontend?

Yes, the publishable key is meant to be public. It only identifies the project. What each signed-in user can do is decided by RLS. The service role key, which bypasses RLS, never goes in the frontend.

Q6. getSession or getUser?

getSession reads the stored session, so it is fast but it trusts local storage. getUser asks the server to verify the token. I use getSession to check whether anyone is signed in and getUser before trusting who they are.

Q7. How do you stop someone giving themselves the admin role?

New accounts are created by a trigger as inactive agents, and the protect_profile_columns trigger stops non-admins changing role, is_active or email, even with a hand-written request.

Q8. Why TanStack Query instead of useEffect and useState?

It caches server data, shares it between components, refetches when it goes stale, retries sensibly, and handles loading and error states. With useEffect I would rewrite all of that by hand, with race conditions.

Q9. How do you keep data fresh after a change?

Every mutation invalidates the queries that read the changed data, including joins. For example, a rename refreshes clients, properties, viewings and the dashboard, because they all show the person's name.

Q10. Why put filters in the URL?

The URL is shareable, survives a refresh and works with the Back button. The params become the query key, so each combination is cached separately.

Q11. Why two Zod schemas per form?

The form works with strings, the database with numbers and null. One schema holds the rules for the form; the second adds a transform for the API. Same rules in both places, conversion defined once.

Q12. Tell me about a bug you fixed.

Use the year-built story: NaN from Number() on an ISO date, turned into null by JSON, so it failed silently. Then the lesson: conversions belong in the schema.

Q13. How does the property approval workflow work?

Every new listing is inserted as pending-approval by the API. Managers and admins see an Approve button; status only changes through its own action, never through the edit form. Next step would be a database trigger so even a direct call cannot publish.

Q14. How do you avoid N+1 queries?

Joins and counts in the same select: viewings(count) for each client, three counted joins for each team member. One request instead of one per row.

Q15. How did you make it fast?

Lazy-loaded pages, skeletons to avoid layout shift, images resized before upload, uploads in parallel, keepPreviousData on paged lists, and database indexes plus policies that run once per query instead of once per row.

Q16. What is the hardest bug to spot in your app?

Silent failures: RLS returning zero rows with no error, and NaN becoming null. Both look like success. I now check results, not just errors.

Q17. How does the demo work without being abused?

The demo password is public on purpose. Triggers lock the demo accounts' role, status, email and password, so visitors can use every permission of the role but cannot lock anyone out.

Q18. What would you improve?

Move account creation to an Edge Function, add a trigger for the approval rule, generate database types to remove casts, and add tests around sign-in and permissions.

Q19. Why Vite and not Next.js for this one?

It is a signed-in dashboard: no public pages to rank in search, so server rendering adds little. Vite gives a fast single-page app, and Supabase is the backend. For a public marketing site I would choose Next.js.

Q20. How is the app accessible?

Semantic HTML, labelled icon buttons, aria-pressed on toggles, role="status" for loading, visible focus rings, and forms that show one clear error per field after it is left.

14. Vocabulary

Term	Meaning
RLS	Row level security: per-row access rules in Postgres
Policy	One RLS rule for a table and an action
Trigger	Database code that runs automatically on insert or update
SECURITY DEFINER	A function that runs with its owner's rights; must be locked down
Publishable key	The public project key; RLS still applies
Service role key	Server-only key that bypasses RLS
Session	The tokens that say who is signed in
Defense in depth	Several layers: UI, validation, database
Silent RLS block	An update that affects zero rows with no error
Query key	The cache identity of a query
Invalidation	Marking cached data stale so it refetches
Dependent query	A query that waits for other data (enabled)
N+1 problem	One query for a list, then one per item
Race condition	Two operations overwriting each other's result
Code splitting	Loading each page's code only when it is opened
Layout shift	Content jumping when data arrives
User enumeration	Learning which emails have accounts from a form's answers

15. Self-test

Answer each out loud in under 45 seconds, without looking.

1. Give the 60-second pitch.
2. Walk through signing in, step by step.
3. Why fetch the user with staleTime: 0 after login?
4. What does ProtectedRoute do, and why Navigate instead of useEffect?
5. Name the three permission layers. Which one is security?
6. What is RLS? Give one of your policies as an example.
7. What happens when RLS blocks an update, and how do you handle it?
8. How can nobody give themselves the admin role?
9. Is the Supabase key in the frontend a problem?
10. getSession vs getUser.
11. How do the demo accounts stay safe?
12. What is a query key? Give one with params.
13. Explain the invalidation rule with the rename bug.
14. Why are filters in the URL?
15. Why two Zod schemas per form?
16. What does action() do?
17. Tell the year-built bug as a STAR story.
18. How does property approval work, and what would make it stronger?
19. How do you avoid N+1 queries?
20. Name three things you would improve.